

1.openvla复现记录

一、参考的一些博客教程

二、仿真复现记录（已跑通）OpenVLA × LIBERO (libero_spatial)

1. 服务器与镜像配置

组云服务器的一些说明

（1）常见的一些租云服务器的平台

（2）租云服务器一般流程

1. 选择算力（显卡类型）——费用

2. 选择服务器带的镜像

3. 开机运行并通过ssh连接到云服务器

2. 工作目录规划（强烈建议）

3. 环境变量（缓存重定向 + headless 渲染）

4. 系统依赖安装（一次性）

5. Conda 环境创建与 PyTorch (cu126)

6. 安装 LIBERO（仿真环境）

6.1 LIBERO 首次运行数据路径配置

7. 安装 OpenVLA eval 封装仓库 (niejnan/OpenVLA)

8. 下载 OpenVLA checkpoint (libero_spatial)

9. 关键坑位与修复记录（踩坑清单）

坑 1：ModuleNotFoundError: No module named 'libero'

坑 2：缺包（依次遇到）

坑 3：FlashAttention2 强制开启但未安装

坑 4：timm 版本不匹配

坑 5：PyTorch 2.6+ 的 torch.load 安全反序列化导致 init_states 读取失败

10. 运行 eval（成功效果）

任务与次数

输出文件位置

11. 性能/速度记录与优化建议

12. 收工/省钱：无卡模式关机

13. 建议备份（下次快速恢复）

复现最短命令清单（下次照抄版）

二、自己复现时遇到的问题

1.使用base模型直接在libero的某个任务上跑遇到了反归一化信息找不到的情况

- (1) 为什么要进行反归一化
- (2) 反归一化键的来源（为什么 Base 模型没有）
- (3) 如何解决base模型没有反归一化信息，导致无法直接在libero任务集上跑？

2.openvla模型测评所需要的环境（踩了AI的坑）

三、对复现的一些疑问和思考

1.对于复现的目标和模型不清晰

1) “复现 OpenVLA”到底在复现什么？

- A. 复现“效果”（最常见、也是你现在的目标）
- B. 复现“训练/微调流程”
- C. 复现“论文方法本身”（包括数据处理、模型结构改动等）

2) 安装官方 openvla/openvla 仓库的目的是什么？用于干什么？

3) “网上的复现版本” niejnan/OpenVLA 仓库又是做什么的？

4) HuggingFace 上的 openvla/openvla-7b-finetuned-libero-spatial 是不是模型文件？

5) 什么是 policy-env glue code？

输入侧 (env → policy)

输出侧 (policy → env)

时序与频率

6) 所以你现在“真正复现”的是什麼？

7) eval.py文件做了什么？环境和模型怎么进行交互的？

8) 我现在使用的openvla/openvla-7b-finetuned-libero-spatial权重模型是针对libero仿真数据训练的?意思...

9) HuggingFace是什么？

10) 对复现结果的一些疑惑

11) 我应该怎么用LORA对其进行微调？

四、对openvla复现相关代码的理解

1.一些概念的理解和代码中的疑问解答

- (1) 什么是processor？
- (2) 什么是flash_attention_2？

1.一句话讲作用

2.他解决了什么问题？

3.他是怎么解决的

(3) SDPA是什么？

1.作用

2.是什么？

(4) lora微调的adapter是如何叠加到原模型上的？ 以及实现原理

1.代码中的实现

2.原理

3.图解

4.公式

(5) libero中的benchmark是什么？

(6) **input中的**是什么作用？

(7) openvla需要机械臂当前的状态信息吗？

(8) Tokenizer 与 dataloader同时并行造成的训练锁死问题

1.为什么会产生锁死现象？

2.如何解决？

2.eval.py解读

(1)代码流程图

3.官方的run_libero_eval.py的代码解读

(1)代码整体功能

(2)带完整中文注释的代码

(3)核心代码执行流程（必看）

1. 配置解析

2. 初始化

3. 加载LIBERO环境

4. 批量评估（核心循环）

5. 结果输出

(4)关键技术细节（OpenVLA专家解读）

(5)总结

五、复现过程中一些补充知识的学习

1.什么是octo模型？

一、Octo 核心定位与背景

二、核心技术架构（Transformer + 扩散策略 + 模块化设计）

1. 输入层：多模态统一 Token 化
2. 主干：因果 Transformer + Readout Token（核心创新）
3. 输出层：扩散策略生成平滑动作序列

三、Octo 三大核心优势（区别于 OpenVLA/RT-1-X）

1. 极致灵活性：跨机器人/传感器/动作空间
2. 动作质量：扩散策略天然平滑、低抖动
3. 开源与易用：完整生态、开箱即用

四、Octo 与 OpenVLA/RT-1-X 关键对比

五、落地价值（对你的机械臂抓取场景）

六、总结

2.什么是FSDP?

一、FSDP 核心定义（大白话版）

二、为什么需要 FSDP？（结合你的场景）

三、FSDP 核心原理（极简拆解）

1. 核心动作：“分片”+“按需通信”
2. 关键区别：对比传统 Data Parallel（DP）
3. 训练流程（以 4 卡训练 Octo 为例）

四、FSDP 在 Octo/OpenVLA 中的应用（对你的价值）

1. Octo/OpenVLA 训练必用 FSDP 的原因
2. 对你的落地价值

五、FSDP 关键配置（极简实操）

六、核心总结

3.什么是ManiSkill2：通用机器人操作的大规模仿真基准

1. 定位与核心目标
2. 核心能力与用途
3. 典型应用场景

4.什么是LIBERO：面向终身/多任务学习的机器人操作基准

1. 定位与核心目标
2. 核心能力与用途
3. 典型应用场景

5.ManiSkill2 vs LIBERO：核心区别对比

6.什么是adapter适配器？

大模型中 Adapter 的核心含义

Adapter 的核心设计与作用

1. 为什么需要 Adapter?
2. Adapter 的典型结构（以LLM为例）
3. 常见的 Adapter 变体
4. Adapter 的使用流程（新手友好）

总结

7.什么是wandb？

（1）参考学习博客

六、base模型直接跑libero的结果

七、openvla的代码结构和其推理测试脚本

- 1.模型代码结构
- 2.推理的测试脚本（图片+指令——>7D的动作）

八、openvla的推理和执行频率篇匹配

- 一、默认执行模式：单步闭环（推理→执行→再推理）
- 二、速度与延迟：满足实时控制，不慢
- 三、如何解决“单步慢”的问题（优化方案）
- 四、一句话总结

九、运行官方测评程序的终端命令

一、参考的一些博客教程

- https://blog.csdn.net/2303_77547168/article/details/156364335?spm=1011.2415.3001.5331小红书博主的openvla的解读和复现教程（没参考）
- <https://photin1a.github.io/posts/20af9ceb.html>openvla的复现教程（组云服务器，比较完善）——主要参

二、仿真复现记录（已跑通）OpenVLA × LIBERO (libero_spatial)

日期：2026-03-08

目标：在云端 4090 上使用 HuggingFace checkpoint `openvla/openvla-7b-finetuned-libero-spatial`，通过 `niejnan/0penVLA` 的 `eval.py` 在 LIBERO 仿真中跑通任务并保存 rollout 视频，看到成功率输出。

1. 服务器与镜像配置

- 平台：彗星云
- GPU：NVIDIA RTX 4090 24GB（1卡）
- 系统：Ubuntu 22.04
- 驱动：nvidia_driver 575.57.08
- Miniconda：py310_23.11.0-2
- 镜像自带：pytorch 2.7.0+cu126 / torchvision 0.22.0+cu126 / torchaudio 2.7.0+cu126
- 磁盘：
 - 系统盘：约 57GB（挂载 `/`）
 - 数据盘：约 295GB（挂载 `/root/data`）
- 计费策略：使用“无卡模式关机”不计实例费用，仅扩容盘少量计费；无卡关机后数据保留 5 天。

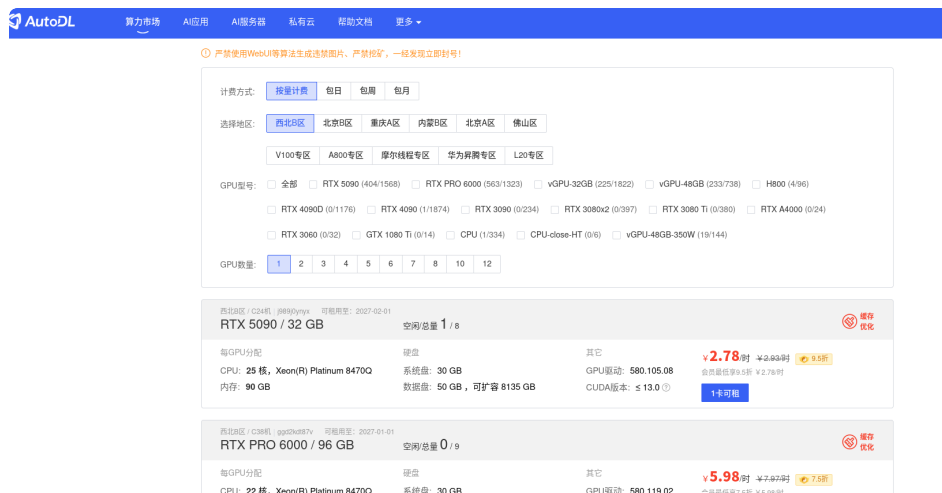
组云服务器的一些说明

(1) 常见的一些租云服务器的平台

- AotuDL：<https://www.autodl.com/market/list>
- 无问芯穹：<https://cloud.infini-ai.com/platform/ai>
- 彗星云：<https://www.huixingyun.com/>（所选）

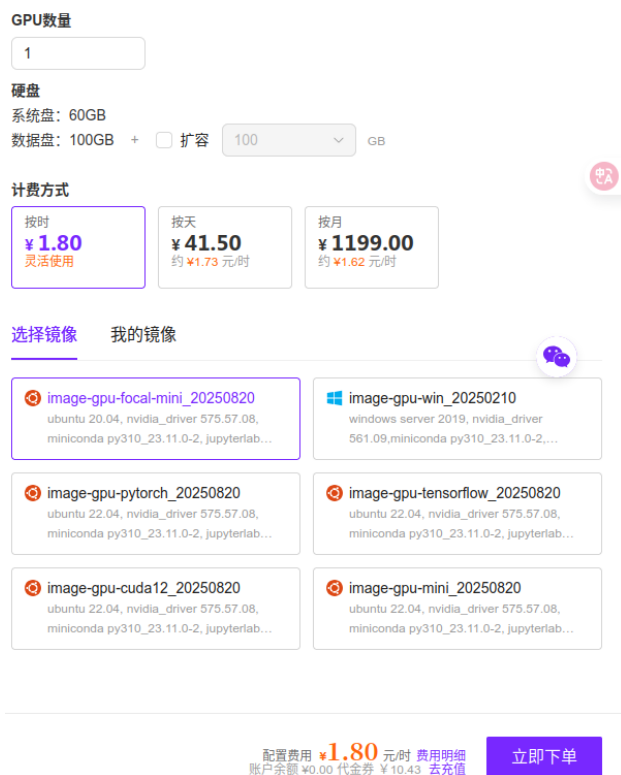
(2) 租云服务器一般流程

1. 选择算力（显卡类型）——费用



2. 选择服务器带的镜像

- 镜像就是租了服务器后，会给你自动配置好一个基础的环境，比如会给你预先安装好ubuntu22.04、conda、pytorch、cuda等等，这个镜像可以选择搭配好的，有的也可以自己定义。



3. 开机运行并通过ssh连接到云服务器

- 一般的云服务器平台都会有任何进行ssh连接的教程文档（以彗星云为例）

新手指引

使用教程

AI 应用教程

操作说明

SSH 远程登录教程

Windows 实例登录教程

云主机登录教程

镜像

我的数据

公共数据

数据上传

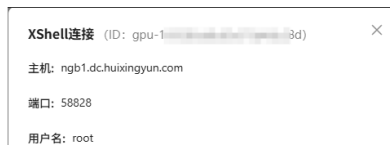
数据下载

学术资源加速

SSH 远程登录教程

Xshell

1、实例开机后，找到SSH登录信息



本目录

Xshell

PyCharm

Visual Studio Code

- 连接成功后，就可以通过终端，来创建环境，拉取代码，训练或者跑模型了

2. 工作目录规划（强烈建议）

所有项目、模型、缓存、视频都放数据盘，避免系统盘写满。

- 工作根目录（数据盘）：
 - `/root/data/opencvla_work`

目录结构建议：

```
1 /root/data/opencvla_work/
2   ├── projects/           # 代码仓库 (OpenVLA、LIBERO)
3   ├── models/            # HF模型checkpoint
4   ├── cache/             # HF / pip cache
5   ├── libero_data/       # LIBERO datasets (自定义路径)
6   ├── backup/            # patch、freeze、记录
7   └── videos/            # (可选) 集中保存视频
```

创建：

```
1 ▾ mkdir -p /root/data/opencvla_work/{projects,models,cache/hf,cache/pip,libero_data,backup}
```

3. 环境变量（缓存重定向 + headless 渲染）

将 HuggingFace / pip 缓存放数据盘；设置 MuJoCo 无头渲染（EGL）。

追加到 `~/.bashrc`：

```
1  cat >> ~/.bashrc <<'EOF'
2  export WORKDIR=/root/data/opencvla_work
3
4  export HF_HOME=$WORKDIR/cache/hf
5  export HUGGINGFACE_HUB_CACHE=$WORKDIR/cache/hf/hub
6  export TRANSFORMERS_CACHE=$WORKDIR/cache/hf/transformers
7  export PIP_CACHE_DIR=$WORKDIR/cache/pip
8
9  export MUJOCO_GL=egl
10 export PYOPENGL_PLATFORM=egl
11
12 # 解决 tokenizers fork 警告 (可选)
13 export TOKENIZERS_PARALLELISM=false
14
15 # 关键：让 OpenVLA 项目下也能 import LIBERO (因为 LIBERO 包结构/导入问题)
16 export PYTHONPATH=$WORKDIR/projects/LIBERO:$PYTHONPATH
17 EOF
18
19 source ~/.bashrc
```

验证：

```
1  echo $WORKDIR
2  echo $MUJOCO_GL
3  echo $PYTHONPATH
```

4. 系统依赖安装（一次性）

用于无头渲染、视频保存、编译依赖等：

```
1  apt-get update
2  apt-get install -y \
3  git wget curl unzip build-essential \
4  libgl1-mesa-glx libgl1-mesa-dri mesa-utils \
5  libegl1-mesa libegl1 \
6  libosmesa6 libosmesa6-dev \
7  libglfw3 libglfw3-dev \
8  ffmpeg xvfb patchelf
```

安装过程中出现 “Daemons using outdated libraries / Which services should be restarted?” 选择默认全选并 OK 即可。

5. Conda 环境创建与 PyTorch (cu126)

新建独立环境，避免污染 base：

```
1 conda create -n openvla_eval python=3.10 -y
2 conda activate openvla_eval
3 pip install -U pip
```

安装与镜像匹配的 torch：

```
1 pip install torch==2.7.0 torchvision==0.22.0 torchaudio==2.7.0 \
2 --index-url https://download.pytorch.org/whl/cu126
3
4 python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"
```

成功标志：`avail True` 且 GPU 为 4090。

6. 安装 LIBERO（仿真环境）

```
1 conda activate openvla_eval
2 cd /root/data/openvla_work/projects
3 git clone https://github.com/Lifelong-Robot-Learning/LIBERO.git
4 cd LIBERO
5 pip install -e .
```

验证：

```
1 python -c "import libero; print('libero import ok')"
```

6.1 LIBERO 首次运行数据路径配置

第一次跑 eval 会提示：

```
Do you want to specify a custom path for the dataset folder? (Y/N)
```

选择 `Y`，输入：

```
/root/data/opencvla_work/libero_data
```

LIBERO 会写配置到：

- `/root/.libero/config.yaml`

7. 安装 OpenVLA eval 封装仓库 (niejnan/OpenVLA)

```
1 conda activate opencvla_eval
2 cd /root/data/opencvla_work/projects
3 git clone https://github.com/niejnan/OpenVLA.git
4 cd OpenVLA
```

安装依赖（跑通所需）：

```
1 pip install transformers==4.40.1 accelerate==0.25.0 peft==0.11.1 \
2 sentencepiece einops jsonlines rich \
3 imageio imageio-ffmpeg \
4 tensorflow-cpu==2.15.0
```

8. 下载 OpenVLA checkpoint (libero_spatial)

下载到数据盘（约 15G）：

```
1 conda activate opencvla_eval
2 pip install -U huggingface_hub
3
4 python - << 'PY'
5 from huggingface_hub import snapshot_download
6 import os
7 repo_id = "opencvla/opencvla-7b-finetuned-libero-spatial"
8 local_dir = "/root/data/opencvla_work/models/opencvla-7b-finetuned-libero-spatial"
9 os.makedirs(local_dir, exist_ok=True)
10 snapshot_download(repo_id=repo_id, local_dir=local_dir, local_dir_use_symlinks=False)
11 print("Downloaded to:", local_dir)
12 PY
```

检查文件齐全（应看到 `config.json` + 分片权重 `model-0000x-of-00004.safetensors`）：

```
1  ls -lh /root/data/opencvla_work/models/opencvla-7b-finetuned-libero-spatial  
   | head -n 30
```

9. 关键坑位与修复记录（踩坑清单）

坑 1: `ModuleNotFoundError: No module named 'libero'`

现象：在 `LIBERO/` 目录能 import，换到 `OpenVLA/` 目录不能 import。

原因：LIBERO 的包结构/导入在该环境下表现异常，最终用 PYTHONPATH 兜底最稳。

修复：在 `~/.bashrc` 加：

```
1  export PYTHONPATH=/root/data/opencvla_work/projects/LIBERO:$PYTHONPATH
```

坑 2: 缺包（依次遇到）

- `imageio` 缺失：

```
1  pip install imageio imageio-ffmpeg
```

- `tensorflow` 缺失：

```
1  pip install tensorflow-cpu==2.15.0
```

- `robosuite` 缺失：

```
1  pip install robosuite
```

- `robosuite` 版本不兼容 LIBERO（缺模块 `SingleArmEnv`）：

解决：降级 `robosuite`（最终稳定）

```
1  pip uninstall -y robosuite  
2  pip install robosuite==1.4.1
```

坑 3: FlashAttention2 强制开启但未安装

报错:

```
ImportError: FlashAttention2 ... flash_attn seems to be not installed
```

修复: 在 `eval.py` 的 `from_pretrained` 中不要强制:

- 删除 `attn_implementation="flash_attention_2"`
或改成:
- `attn_implementation="sdpa"`

(本次复现采用“不用 flash-attn”跑通。)

坑 4: timm 版本不匹配

报错:

```
NotImplementedError: TIMM Version must be >= 0.9.10 and < 1.0.0
```

修复:

```
1 pip uninstall -y timm
2 pip install timm==0.9.10
```

坑 5: PyTorch 2.6+ 的 `torch.load` 安全反序列化导致 `init_states` 读取失败

报错 (关键):

```
_pickle.UnpicklingError: Weights only load failed...
```

原因: PyTorch 2.6+ 默认 `torch.load(weights_only=True)`, LIBERO `init_states` 包含 `numpy` 对象被拦截。

修复: 修改 LIBERO 源码:

文件:

```
/root/data/openvla_work/projects/LIBERO0/libero/libero/benchmark/__init__.py
```

将：

```
1 init_states = torch.load(init_states_path)
```

改为：

```
1 init_states = torch.load(init_states_path, weights_only=False)
```

10. 运行 eval（成功效果）

运行：

```
1 conda activate openvla_eval
2 cd /root/data/openvla_work/projects/OpenVLA
3 python eval.py
```

成功标志：

- 模型加载分片完成： Loading checkpoint shards: 100%
- 输出：
 - Logging to local log file: ...
 - Task suite: libero_spatial, task_num: 10
- 逐任务打印任务描述：
 - 如： Task: pick up the black bowl ... place it on the plate
- 每条 episode 输出成功率统计：
 - Success: True/False
 - # episodes completed so far: ...
- 保存 MP4 视频：
 - Saved rollout MP4 at path ./rollouts/YYYY_MM_DD/...episode=...success=...task=...mp4

任务与次数

- suite: libero_spatial
- 任务数: 10 (task_num: 10)
- 每任务 trials: config.num_trials_per_task (本次为 2)

- 总 episodes: $10 \times 2 = 20$

输出文件位置

- 视频默认在:
 - `/root/data/opencvla_work/projects/OpenVLA/rollouts/2026_03_08/*.mp4`
 - 日志在:
 - `/root/data/opencvla_work/projects/OpenVLA/experiments/logs/*.txt`
 - LIBERO 配置:
 - `/root/.libero/config.yaml`
-

11. 性能/速度记录与优化建议

现象：每个 episode 可能较慢（~90s/episode）。主要原因通常是：

- 每 step 大模型推理 + 环境渲染 + 保存视频帧（深拷贝 + 编码）

优化建议（按收益优先级）：

1. 降低 `utils.get_libero_env(... resolution=768)` 的分辨率到 256 或 224
 2. 关闭视频保存或每 N 步保存一帧
 3. 如果显存足够，关闭 bnb 8bit/4bit（有时 8bit 不更快，只是省显存）
 4. 使用 `TOKENIZERS_PARALLELISM=false` 关闭 tokenizers 并行 fork 警告
-

12. 收工/省钱：无卡模式关机

- 使用“无卡模式关机”释放 GPU/CPU/内存，不计实例费用
 - 注意：无卡关机数据保留 5 天；超期自动清除不可恢复
 - 无卡关机期间仅扩容数据盘计费：0.0004 元/GB/小时
-

13. 建议备份（下次快速恢复）

建议保存关键改动和环境信息：

```
1  mkdir -p /root/data/opencvla_work/backup
2
3  # 环境依赖
4  conda activate opencvla_eval
5  pip freeze > /root/data/opencvla_work/backup/pip_freeze_opencvla_eval.txt
6
7  # 改动补丁（可选）
8  cd /root/data/opencvla_work/projects/OpenVLA
9  git diff > /root/data/opencvla_work/backup/OpenVLA_changes.patch || true
10
11 cd /root/data/opencvla_work/projects/LIBERO
12 git diff > /root/data/opencvla_work/backup/LIBERO_changes.patch || true
```

复现最短命令清单（下次照抄版）

1. 装系统依赖（第4节）
2. conda env + torch cu126（第5节）
3. clone + install LIBERO（第6节）
4. clone OpenVLA (niejnan) + pip deps（第7节）
5. 下载 HF checkpoint（第8节）
6. 修改：LIBERO `torch.load(weights_only=False)`（坑5）
7. 修改：eval.py 不强制 flash-attn（坑3）
8. robosuite 降级到 1.4.1（坑2）
9. timm pin 到 0.9.10（坑4）
10. `python eval.py` 跑 libre_spatial（10任务×2次=20 episode）并保存 mp4

如果你愿意，我还可以把这份记录进一步升级成：

- 一个 `setup_opencvla_eval.sh`（一键安装依赖 + 打补丁）
- 一个 `run_eval.sh`（自动设置环境变量、自动创建目录、直接开跑）

这样你下次租机基本只需要“复制粘贴两条命令”就能恢复到可跑状态。

二、自己复现时遇到的问题

1.使用base模型直接在libero的某个任务上跑遇到了反归一化信息找不到的情况

(1) 为什么要进行反归一化

OpenVLA 模型在训练阶段会对动作做归一化（Normalization），目的是让不同维度、不同量级的动作特征（如机械臂位移、夹爪开合度）缩放到统一范围（通常是 `[-1, 1]`），提升模型训练的稳定性和收敛速度；而推理阶段，模型输出的是「归一化后的动作」，必须通过反归一化还原到「仿真环境能识别的原始物理空间」，否则动作数值无意义（甚至会导致机器人执行错误操作）。

(2) 反归一化键的来源（为什么 Base 模型没有）

`model.norm_stats` 是模型在训练 / 微调阶段从数据集统计信息中保存的动作归一化参数（如 `q0`、`1/q99`、`mean/std` 等），其键名对应训练数据集的名称：

1. 官方微调后的 checkpoint：官方在 `libero_spatial` 数据集上微调时，会将该数据集的动作统计信息（如每个维度的极值、分位数）保存到 `dataset_statistics.json`，加载模型时会把这些信息注入 `model.norm_stats`，因此 `norm_stats` 中会有 `libero_spatial` 键。
2. 纯 Base 模型（`openvla-7b`）：Base 模型的预训练数据是通用多模态数据（非 LIBERO 机器人数据），其 `norm_stats` 中仅包含预训练数据集的键（如 `bridge_orig`、`language_table` 等），完全没有 `libero_spatial` 相关的键。

(3) 如何解决base模型没有反归一化信息，导致无法直接在libero任务集上跑？

直接把官方微调好的模型中的`dataset_statistics.json`复制到base模型中，再把`dataset_statistics.json`中的统计信息注入到`model.norm_stats["libero_spatial"]`中，即可完成动作的反归一化。

```

# ===== 校验OpenVLA模型的反归一化统计信息 =====
# ===== 新增: Base 模型适配 LIBERO 反归一化 =====
# 区分 Base 模型和微调模型
is_base_model = "openvla-7b" in str(cfg.pretrained_checkpoint) and "finetuned" not in str(cfg.pretrained_checkpoint)
if is_base_model:
    # 方案 1: 注入 LIBERO 统计信息 (推荐)
    try:
        import json
        # 替换为你的 dataset_statistics.json 路径
        stats_path = cfg.pretrained_checkpoint + "/dataset_statistics.json"
        with open(stats_path, "r") as f:
            all_stats = json.load(f)
        if "libero_spatial" in all_stats:
            model.norm_stats["libero_spatial"] = all_stats["libero_spatial"]
            print("✅ 已为 Base 模型注入 libero_spatial 统计信息")
        else:
            # 方案 2: 降级复用通用键
            cfg.unnorm_key = "bridge_orig"
            print("⚠️ 未找到 libero_spatial 统计信息, 复用 bridge_orig 键")
    except FileNotFoundError:
        # 方案 2: 降级复用通用键
        cfg.unnorm_key = "bridge_orig"
        print("⚠️ 未找到统计文件, 复用 bridge_orig 键")

# 最终检查 (兼容 Base/微调模型)
assert cfg.unnorm_key in model.norm_stats, f"动作反归一化键 {cfg.unnorm_key} 不存在于模型统计信息中! "

```

注意这里使用数据集自己微调训练的反归一化统计信息的key的名称不一定是libero_spatial有可能是其他名称, 请点击查看

2.openvla模型测评所需要的环境（踩了AI的坑）

- AI给的安装顺序，是针对的不安装openvla的官方仓库的依赖，而是自己手动一个个安装。

1.只需要先克隆openvla的官方仓库，然后执行下面这句，安装openvla所需要的官方依赖（基本上包含了所有依赖，pytorch等等）

- 这句代码-e表示的是可编辑安装，也就是在你安装之后，如果又修改了这个包的源码，是直接就可以链接到修改的地方，不需要修改一次安装一次，适合开发者。
- 执行这句代码，就会寻找当前目录下的pyproject.toml或者setup.py文件，根据里面描述的依赖，进行安装

```

1  pip install -e .

```

2.如果需要用到libero，就先克隆libero仓库，同样也是安装依赖

```

1  git clone https://github.com/Lifelong-Robot-Learning/LIBERO.git
2  cd LIBERO
3  pip install -e .

```

3.此时还需要回到openvla目录，安装一些对libero的专属依赖

```
▼ Bash |
1 # 回到 OpenVLA 根目录，安装 LIBERO 专属依赖
2 cd /path/to/your/openvla
3 pip install -r experiments/robot/libero/libero_requirements.txt
```

4.安装完这些基础环境后，就需要根据AI安装用于无头渲染、视频保存、编译依赖(可能还需要下载 mujoco)

5.终极大法，就是安装完基础环境就先去尝试运行，缺什么依赖就安装什么

三、对复现的一些疑问和思考

1.对于复现的目标和模型不清晰

1) “复现 OpenVLA”到底在复现什么？

通常有三种层级（从易到难）：

A. 复现“效果”（最常见、也是你现在的目标）

- 含义：用作者/官方发布的 训练好 checkpoint，在标准任务（如 LIBERO）里跑评测/演示，看到成功率或可视化结果。
- 你现在做的就是这个：不从头训练模型，只是复现它在仿真任务上的表现。

B. 复现“训练/微调流程”

- 含义：用相同的数据、相同的训练脚本，把模型从 base（或别的起点）训练到类似性能。
- 成本更高：需要大量算力/数据/时间。

C. 复现“论文方法本身”（包括数据处理、模型结构改动等）

- 含义：把论文里的完整方法链条（数据→模型→训练→评测）都重走一遍。

你的文章路线明显属于 **A：复现效果**。

2) 安装官方 `openvla/openvla` 仓库的目的是什么？用于干什么？

把它理解成：“官方实现/官方生态的总仓库”，通常包含：

1. 模型相关的代码（如何构建模型、如何tokenize、如何把视觉和语言编码成模型输入）
2. 训练/微调脚本（你未来想微调时会用到）
3. 与任务环境相关的工具（例如 LIBERO 的 requirements、数据处理、评测入口等）
4. 依赖版本的权威参考（torch、transformers、flash-attn、tensorflow 等该用什么版本更兼容）

在你这篇文章里，作者安装官方仓库主要起到两点作用：

- 照着官方依赖把环境配齐（比如 requirements、flash-attn、libero_requirements）
- 为后续跑仿真/下载模型做准备（虽然严格说下载模型不一定非得装官方仓库，但文章按这个路径走）

关键：官方 `openvla` 仓库不等于“模型本体”。
它是“代码”，而模型权重在 HuggingFace 上。

3) “网上的复现版本” `niejnan/OpenVLA` 仓库又是做什么的？

它更像一个第三方封装好的“开箱即用运行器”，专门把“OpenVLA 模型 + LIBERO 环境 + 评测循环”这三件事粘起来，让你能：

- 指定一个 checkpoint 路径
- 选择一个 task suite (libero_spatial 等)
- 一条命令跑 `eval.py`
- 自动完成：加载模型 → reset 环境 → 循环预测动作 → step 环境 → 统计成功率/保存视频

也就是说，它主要提供的是评测/演示管线，对新手很友好。

和官方仓库相比：

- 官方仓库更“完整/研究导向”（训练、数据、更多组件）
- `niejnan/OpenVLA` 更“工程封装/跑通导向”（快速让你看到机械臂动起来）

4) HuggingFace 上的 `openvla/openvla-7b-finetuned-l` `ibero-spatial` 是不是模型文件？

基本可以把它理解为：“一个可以直接用的预训练/微调好的模型包”。

它通常包含两类东西（具体看仓库文件列表，但一般如此）：

1. 权重文件

- 例如 `model.safetensors` 或 `pytorch_model.bin`（可能被分片）
- 这是模型学到的参数（最关键、体积最大）

2. 配置与Tokenizer等

- `config.json`：模型结构配置（多少层、hidden size 等）
- `tokenizer.json` / `tokenizer.model`：分词器
- 可能还有 `preprocessor_config.json` 等

所以你问“包括模型结构和权重吗？”——更准确地说：

- 结构的“定义代码”在 Python 代码里（`transformers/openvla` 相关实现）
- 结构的“超参数配置”在 `config.json`
- 权重在 `safetensors/bin`

这三者结合起来才能真正加载并运行。

5) 什么是 `policy-env glue code`？

这是机器人学习里很常用的说法，直译就是“策略（policy）和环境（env）之间的胶水代码”。

因为模型（policy）和仿真环境（env）天生“接口不一致”，必须写一层东西把它们对齐。典型要做的事情包括：

输入侧（env → policy）

- 从 env observation 里取出相机图像（可能多相机、多帧）
- resize/crop/归一化
- 拼上语言指令（instruction）
- 整理成模型需要的张量格式（batch维度、dtype、device）

输出侧（policy → env）

- 模型输出的 action 可能是：
 - 末端位姿增量（dx,dy,dz, droll,dpitch,dyaw）
 - 关节速度
 - 或离散token需要再解码
- 你要把它转换为 env 接受的 action space：
 - 维度要一致
 - 范围要正确（clip、scale）
 - gripper 开合要映射到正确的通道

时序与频率

- 模型每预测一次 action，环境 step 一次还是多次？
- 是否要 action smoothing？
- 最大步数、终止条件、成功判定如何记录？

这些对齐工作就是 glue code。写不好会出现：

- 机械臂乱甩（action尺度错）
- 基本不动（动作被clip成0或频率不对）
- 总抓空（相机输入/坐标系对不上）

你文章里用 `niejnan/OpenVLA` 的最大好处就是：别人已经把这层 glue code 写好了，你只要改 checkpoint 路径和 suite 名称就能跑。

6) 所以你现在“真正复现”的是什么？

按照文章路线，你复现的是：

- OpenVLA 在 LIBERO (libero_spatial suite) 上的推理与闭环控制表现

- 使用的模型是 HF 上发布的 `finetuned-libero-spatial checkpoint`
- 使用的运行管线主要来自 `niejnan/OpenVLA`（封装好的评测脚本）

而不是从头训练 OpenVLA。

7) eval.py文件做了什么？环境和模型怎么进行交互的？

8) 我现在使用的openvla/openvla-7b-finetuned-libero-spatial权重模型是针对libero仿真数据训练的？意思是还有其他权重模型？

- 对我当前用的openvla/openvla-7b-finetuned-libero-spatial模型文件是base模型针对libero-spatial任务集上进行微调过的。
- **base（未针对 LIBERO 微调）**：更通用，但在特定 benchmark 上不一定强。
 - `openvla/openvla-7b`，OpenVLA 的 7B 基座/通用模型（base checkpoint），不专门对 LIBERO 某个 suite 做最后的强化微调，更像“基础能力”，可以再针对下游任务做 finetune（LoRA 等）
- **finetuned（针对某个 suite 微调）**：在对应 suite 上效果好，适合你做仿真评测（**finetuned模型微调的意思**）
 - `openvla/openvla-7b-finetuned-libero-spatial`，在 LIBERO 的 **spatial suite** 上做过 finetune 的 7B 模型。除此之外，还有 `openvla/openvla-7b-finetuned-libero-object`，在 LIBERO 的 **object suite** 上 finetune 的模型。
- 我如果想要尝试lora微调，就得对比base模型在某个任务上的效果，然后自己基于base模型对这个任务进行LORA微调

9) HuggingFace是什么？

HuggingFace（常简称 HF）可以理解成 AI 领域的“模型与数据的应用商店 + 版本管理平台”，最核心的是：

- **Model Hub**: 托管模型（权重文件、配置文件、tokenizer、以及必要的自定义代码）
- **Dataset Hub**: 托管数据集
- **Transformers 库**: 最常用的模型加载与推理/训练框架之一（你在代码里用的 `AutoModel...from_pretrained` 就是它）
- **huggingface_hub**: 下载/缓存/版本管理工具（你用 `snapshot_download` 下载模型）

你现在之所以能在服务器上“一条命令加载 OpenVLA 模型”，就是因为：

- 权重与配置放在 HuggingFace
- Transformers 知道如何从 HuggingFace 拉取并加载
- `trust_remote_code=True` 允许使用模型仓库里自定义的模型实现代码（例如你日志里的 `modeling_prismatic.py`）

10) 对复现结果的一些疑惑

11) 我应该怎么用LORA对其进行微调？

四、对openvla复现相关代码的理解

1.一些概念的理解和代码中的疑问解答

(1) 什么是 `processor` ？

`AutoProcessor` 通常把 文本 prompt 和 图像 变成模型需要的张量：

- 文本 → token ids / attention mask
- 图像 → pixel_values（归一化、resize 等）

你这里 `processor` 的调用方式是：


```
1 Python
2
3 inputs = processor(prompt, Image.fromarray(img).convert("RGB"))
```

说明这个 processor 支持“文本+图像”的联合输入。

(2) 什么是flash_attention_2?

1.一句话讲作用

Flash-Attention 2 是一种对 Transformer 注意力机制的显存优化加速算法，能让大模型训练 / 推理更快、更省显存、支持更长文本 / 图像序列。

它不改变模型精度、不改变模型结构，只是让 Attention 计算更高效。

2.他解决了什么问题?

标准 Transformer Attention 有两个致命问题：

1. 显存占用极大（复杂度 $O(N^2)$ ）
2. 速度慢

OpenVLA 是视觉 - 语言 - 动作模型，图像会被分成大量 patch，导致输入序列长度非常长。

3.他是怎么解决的

它不存储完整的注意力权重矩阵，而是分块计算，把中间结果存在高速显存中，大幅降低内存访问开销。

(3) SDPA是什么?

1.作用

指定模型使用 PyTorch 官方的 SDPA (Scaled Dot Product Attention) 高效注意力实现，替代手动写的普通 Attention，实现自动速度和显存优化。

OpenVLA 的 ViT 视觉编码器和 LLM 语言模型都有大量的 Attention 层。我在加载模型时设置 `attn_implementation="sdpa"`，让 PyTorch 自动启用高效原生注意力，在不修改模型结构、不影响精度的前提下，加快训练速度、降低显存占用，让 7B 模型在单张消费级 GPU 上也能稳定运行 LoRA 微调。

2.是什么？

- SDPA = Scaled Dot Product Attention
- 是 PyTorch 2.0+ 原生内置的高效注意力层
- 相当于 PyTorch 官方版的“轻量 Flash Attention”
- 不需要额外安装 flash-attn 库
- 不需要改代码，开箱即用加速

(4) lora微调的adapter是如何叠加到原模型上的？ 以及实现原理

面试回答 Plain Text

1 LoRA 的核心思想是不在主干模型上修改权重，而是在 Transformer 的线性层旁插入两个低秩分解矩阵 A 和 B。

2 在加载时，通过 `PeftModel.from_pretrained` 将训练好的 LoRA 矩阵注册到模型的对应层。

3 前向传播时，模型的原始输出会加上 LoRA 产生的增量输出，实现高效的适配。

4 这种方式只训练极少参数，却能让模型学习到下游任务知识，同时不破坏原模型的泛化能力。

1.代码中的实现

Python

1 `model = PeftModel.from_pretrained(model, config.lora_checkpoint)`

- 核心函数：从文件中加载训练好的 LoRA 权重
- 第 1 个参数 `model`：你的基础模型（OpenVLA/LLaMA/ViT 主干）
- 第 2 个参数 `lora_checkpoint`：LoRA 权重文件夹
- 返回：基础模型 + LoRA 适配器绑定在一起的新模型

PEFT 库自动做了 4 件事：

1. 遍历模型所有线性层（Linear 层）
2. 给每个层注册 LoRA 的 A、B 矩阵
3. 把预训练好的数值赋值给 A、B
4. 前向传播时自动执行： $Wx + BAx$

这就叫 Adapter 叠加（Adapter Fusion）

2.原理

LoRA 不会修改原模型的权重，而是在模型的线性层旁，插入两个小的低秩矩阵 A 和 B。前向传播时，原始输出 + (A×B) 的增量，一起输出。

3.图解

```
1  原始线性层：
2      x → [ 主干层 W ] → 输出
3
4  加入 LoRA 后：
5      x → [ 主干层 W ] → 原始输出
6      x → [ A ] → [ B ] → 增量 Δoutput
7  最终输出 = 原始输出 + Δoutput
```

4.公式

$$h = Wx + BAx$$

- W = 主干模型权重（冻结，不训练）
- A, B = LoRA 小矩阵（训练）
- BA = 低秩分解，参数极少

(5) libero中的benchmark是什么？

```
1  benchmark_dict = benchmark.get_benchmark_dict()
2  # 获取 LIBERO 里所有【标准任务套件】的名称与对应环境
```

benchmark 就是 → 一套「标准机器人任务测试集」它的作用是：统一任务环境、统一评估规则、统一算任务成功率，用来公平对比模型好坏。

在 LIBERO 里：

- benchmark = 一套固定的机器人操作任务套件
- 例如： `libero_spatial` 、 `libero_object` 、 `libero_10` 、 `libero_90`

面试回答：

LIBERO 是一个具身智能的标准基准（benchmark），

我用来评估我微调后的 OpenVLA 模型的任务执行能力。

我通过 benchmark 加载指定任务套件（如 `libero_spatial`），让模型执行语言引导的机器人操作，最终统计任务成功率，用来验证 LoRA 微调确实有效提升了模型性能。

(6) `**input`中的`**`是什么作用？

`**` 是 Python 的「字典解包运算符」作用是：

- 把一个字典里的所有键值对，自动拆成「key=value」形式传给函数当参数。

▼ 解释：

Python |

```
1  假设你的 inputs 长这样（是一个字典）：
2  inputs = {
3      "image": 图片张量,
4      "input_ids": text token,
5      "attention_mask": mask
6  }
7
8  当你写：
9  model.predict_action(**inputs)
10
11 Python 自动帮你变成：
12 model.predict_action(
13     image=图片张量,
14     input_ids=text token,
15     attention_mask=mask
16 )
```

- 模型输入有很多项（image、input_ids、attention_mask...）
- 这些输入在代码里已经被打包成了一个字典 `inputs`
- 用 `**` 可以不用一个个手写参数，代码更干净、通用、可扩展

(7) openvla需要机械臂当前的状态信息吗？

不需要

Plain Text

- 1 传统 RL 算法缺乏视觉感知能力，必须依赖外部状态输入；
- 2 而 OpenVLA 这类大模型具备**端到端视觉理解能力**，可以直接从图像中学习策略，
- 3 不需要显式的状态向量，鲁棒性反而更强，更贴近现实机器人部署场景。
- 4
- 5 在 LIBERO 这种标准化环境里，纯视觉推理**非常稳定、非常准确**，
- 6 完全能支撑策略学习和任务成功。

(8) Tokenizer 与 dataloader同时并行造成的训练锁死问题

1.为什么会产生锁死现象？

HuggingFace 的 Tokenizer 默认会自动开启多线程来加速文本编码。

但同时：

- PyTorch 的 `DataLoader` 也会开多进程 / 多线程加载数据
- 两个库同时抢线程资源
- 就会出现 互相等待、卡死、程序不动 → 这就是 死锁 (Deadlock)

现象：

- 训练一开始就卡住
- 不报错、不运行、GPU 占用为 0
- 控制台不动

这是 PyTorch + DataLoader + Tokenizer 一起使用时的经典 BUG 解决方案。

2.如何解决？

Python

```
1 os.environ["TOKENIZERS_PARALLELISM"] = "false"
```

强制让 HuggingFace Tokenizer 只使用单线程，关闭多线程并行，避免训练时程序卡死、死锁、崩溃。

它设置一个环境变量，告诉 Tokenizer：

不许使用多线程，老老实实单线程编码！

2.eval.py解读

- 从 HuggingFace checkpoint 加载 OpenVLA 模型 + processor
- 从 LIBERO benchmark 里依次取出 libero_spatial 的 10 个任务
- 每个任务跑 `num_trials_per_task` 次 episode
- 每个 episode:
 - reset 到固定 init state
 - 每个时间步取相机图像 + 任务语言描述 → 构造 prompt
 - 用 `model.predict_action()` 预测下一步动作 (7维)
 - 对夹爪维度做转换 (0/1 → -1/+1 等)
 - `env.step(action)` 执行动作
 - 保存视频帧
- episode 结束后保存 mp4, 打印累计成功率

(1)代码流程图

```

1  启动脚本
2    └─ 创建 Config、设置 DEVICE
3
4  eval_libero()
5    └─ 断言: checkpoint 必须有; 8bit 和 4bit 不能同时开
6    └─ 加载 VLA 模型 (AutoModelForVision2Seq.from_pretrained)
7    └─ (可选) 加载 LoRA 权重 (PeftModel.from_pretrained)
8    └─ (可选) 把模型搬到 GPU (量化时通常不用手动 .to)
9    └─ 设置 task_suite_name=libero_spatial, unnorm_key=libero_spatial
10   └─ 断言: 模型里有这套 norm_stats (用于反归一化动作)
11   └─ 加载 processor (AutoProcessor.from_pretrained)
12
13   └─ 初始化日志文件 experiments/logs/...
14   └─ 从 libero.benchmark 取 benchmark_dict
15   └─ 构造 task_suite = libero_spatial()
16   └─ 读取 suite 任务数 n_tasks
17
18   └─ for task_id in [0..n_tasks-1]:
19       └─ task = task_suite.get_task(task_id)
20       └─ initial_states = task_suite.get_task_init_states(task_id)
21       └─ env, task_description = get_libero_env(task, resolution=768)
22       └─ for episode_idx in [0..num_trials_per_task-1]:
23           └─ env.reset()
24           └─ obs = env.set_init_state(initial_states[episode_idx])
25           └─ t=0, replay_images=[]
26           └─ while t < max_steps:
27               └─ if t < 10: 先step一个“静止动作”让物体落稳
28                   └─ img = obs["agentview_image"]
29                   └─ img 翻转180度
30                   └─ replay_images.append(img)
31                   └─ img resize -> 224x224 (用tf的lanczos3)
32                   └─ prompt = "In: ... task_description ...\nOut:"
33                   └─ inputs = processor(prompt, PIL_image).to(device,bf16)
34                   └─ action = model.predict_action(..., unnorm_key, do_sample=False)
35
36               └─ 夹爪维度处理: 0/1 -> -1/+1 -> sign -> 反号
37               └─ obs, reward, done, info = env.step(action)
38               └─ if done: 统计成功, break
39               └─ t++
40
41           └─ episode结束: task_episodes/total_episodes++
42           └─ save_rollout_video(replay_images,...)
43           └─ 打印/写日志: success(done)、成功率等
44
45   └─ 每个task结束: 打印/写该task成功率、总成功率

```

3.官方的run_libero_eval.py的代码解读

我是OpenVLA专家，我将为你逐行添加中文详细注释，并完整解释这段代码的核心功能、执行流程和技术细节。

(1)代码整体功能

这是OpenVLA模型在LIBERO机器人仿真环境中的评估脚本，核心作用：

1. 加载训练好的OpenVLA模型
2. 在LIBERO的5种任务集（空间/物体/目标/10任务/90任务）中自动执行机器人操作
3. 统计任务成功率、记录日志、生成演示视频
4. 支持Wandb在线可视化监控

(2)带完整中文注释的代码


```

1  """
2  run_libero_eval.py
3
4  在 LIBERO 仿真环境中运行模型进行评估。
5
6  使用方法:
7      # OpenVLA:
8      # 重要提示: 如果模型是使用数据增强训练的, 请设置 center_crop=True
9      python experiments/robot/libero/run_libero_eval.py \
10         --model_family openvla \
11         --pretrained_checkpoint <CHECKPOINT_PATH> \
12         --task_suite_name [ libero_spatial | libero_object | libero_goal
| libero_10 | libero_90 ] \
13         --center_crop [ True | False ] \
14         --run_id_note <OPTIONAL TAG TO INSERT INTO RUN ID FOR LOGGING> \
15         --use_wandb [ True | False ] \
16         --wandb_project <PROJECT> \
17         --wandb_entity <ENTITY>
18  """
19
20  # ===== 1. 导入依赖库 =====
21  import os
22  import sys
23  from dataclasses import dataclass # 用于定义配置类
24  from pathlib import Path
25  from typing import Optional, Union
26
27  import draccus # 命令行参数解析库
28  import numpy as np
29  import tqdm # 进度条工具
30  from libero.libero import benchmark # LIBERO 仿真环境基准库
31
32  import wandb # 在线日志可视化工具
33
34  # 将上级目录添加到Python路径, 确保能导入实验相关代码
35  sys.path.append("../..")
36
37  # 导入LIBERO环境工具函数: 获取环境、图像、动作、保存视频等
38  from experiments.robot.libero.libero_utils import (
39      get_libero_dummy_action,
40      get_libero_env,
41      get_libero_image,
42      quat2axisangle,
43      save_rollout_video,
44  )

```

```

45
46 # 导入OpenVLA专用工具：获取模型处理器
47 from experiments.robot.openvla_utils import get_processor
48
49 # 导入通用机器人工具函数：加载模型、执行动作、随机种子等
50 from experiments.robot.robot_utils import (
51     DATE_TIME,
52     get_action,
53     get_image_resize_size,
54     get_model,
55     invert_gripper_action,
56     normalize_gripper_action,
57     set_seed_everywhere,
58 )
59
60
61 # ===== 2. 定义评估配置类（所有参数都在这里） =====
62 ===
63 @dataclass
64 class GenerateConfig:
65     # fmt: off
66
67     #####
68     #####
69     # 模型相关参数
70     #####
71     #####
72     model_family: str = "openvla" # 模型类型，固定为open
73     vla
74     pretrained_checkpoint: Union[str, Path] = "" # 训练好的模型权重路径
75     (必须手动指定)
76     load_in_8bit: bool = False # OpenVLA专用：是否以8
77     位量化加载模型（节省显存）
78     load_in_4bit: bool = False # OpenVLA专用：是否以4
79     位量化加载模型（节省显存）
80
81     center_crop: bool = True # 是否中心裁剪：模型用
82     随机裁剪增强训练时必须设为True
83
84     #####
85     #####
86     # LIBERO 环境相关参数
87     #####
88     #####
89     task_suite_name: str = "libero_spatial" # 任务集名称：可选libe
90     ro_spatial/object/goal/10/90
91     num_steps_wait: int = 10 # 仿真初始等待步数（让
92     物体稳定下落）

```

```

81     num_trials_per_task: int = 50                                # 每个任务执行多少次尝
    试（评估更准确）
82
83     #####
84     # 日志与工具参数
85     #####
86     run_id_note: Optional[str] = None                            # 运行ID备注（用于区分
    不同实验）
87     local_log_dir: str = "./experiments/logs"                    # 本地日志保存文件夹
88
89     use_wandb: bool = False                                       # 是否使用Wandb在线日
    志
90     wandb_project: str = "YOUR_WANDB_PROJECT"                    # Wandb项目名（手动修
    改）
91     wandb_entity: str = "YOUR_WANDB_ENTITY"                      # Wandb用户名/团队名
    （手动修改）
92
93     seed: int = 7                                                 # 随机种子（保证实验可
    复现）
94
95     # fmt: on
96
97
98     # ===== 3. 核心评估主函数 =====
99     # draccus装饰器：自动解析命令行参数为配置对象
100    @draccus.wrap()
101    def eval_libero(cfg: GenerateConfig) -> None:
102        # ===== 参数合法性检查 =====
103        assert cfg.pretrained_checkpoint is not None, "cfg.pretrained_checkpo
    int 不能为空！"
104        # 如果模型路径包含图像增强，必须开启中心裁剪
105        if "image_aug" in cfg.pretrained_checkpoint:
106            assert cfg.center_crop, "模型使用图像增强训练，必须设置 center_crop=Tr
    ue！"
107        # 不能同时使用8位和4位量化
108        assert not (cfg.load_in_8bit and cfg.load_in_4bit), "不能同时使用8位和4
    位量化！"
109
110        # ===== 初始化：固定随机种子 =====
111        set_seed_everywhere(cfg.seed)
112
113        # ===== OpenVLA 专用设置 =====
114        # 设置动作反归一化的键（根据任务集名称）
115        cfg.unnorm_key = cfg.task_suite_name
116
117        # ===== 加载模型 =====

```

```

118     model = get_model(cfg)
119
120     # ===== 校验OpenVLA模型的归一化统计信息 =====
121     ==
122     if cfg.model_family == "openvla":
123         # 兼容特殊数据集命名 (带_no_noops后缀)
124         if cfg.unnorm_key not in model.norm_stats and f"{cfg.unnorm_key}_
no_noops" in model.norm_stats:
125             cfg.unnorm_key = f"{cfg.unnorm_key}_no_noops"
126             # 确保反归一化键存在
127             assert cfg.unnorm_key in model.norm_stats, f"动作反归一化键 {cfg.un
norm_key} 不存在于模型统计信息中! "
128
129     # ===== 加载OpenVLA处理器 (图像+文本预处理) =====
130     ==
131     processor = None
132     if cfg.model_family == "openvla":
133         processor = get_processor(cfg)
134
135     # ===== 初始化本地日志 =====
136     # 生成唯一运行ID: 评估-任务集-模型-时间
137     run_id = f"EVAL-{cfg.task_suite_name}-{cfg.model_family}-{DATE_TIME}"
138     if cfg.run_id_note is not None:
139         run_id += f"--{cfg.run_id_note}"
140     # 创建日志文件夹
141     os.makedirs(cfg.local_log_dir, exist_ok=True)
142     local_log_filepath = os.path.join(cfg.local_log_dir, run_id + ".txt")
143     log_file = open(local_log_filepath, "w")
144     print(f"本地日志文件路径: {local_log_filepath}")
145
146     # ===== 初始化Wandb在线日志 =====
147     if cfg.use_wandb:
148         wandb.init(
149             entity=cfg.wandb_entity,
150             project=cfg.wandb_project,
151             name=run_id,
152         )
153
154     # ===== 初始化LIBERO任务集 =====
155     benchmark_dict = benchmark.get_benchmark_dict()
156     task_suite = benchmark_dict[cfg.task_suite_name]() # 实例化指定任务集
157     num_tasks_in_suite = task_suite.n_tasks # 获取任务集中的任务总数
158     print(f"当前任务集: {cfg.task_suite_name}")
159     log_file.write(f"当前任务集: {cfg.task_suite_name}\n")
160
161     # 获取模型需要的图像尺寸
162     resize_size = get_image_resize_size(cfg)

```

```

162 # ===== 开始全局评估 =====
163 total_episodes, total_successes = 0, 0 # 总尝试次数、总成功次数
164
165 # 遍历任务集中的每一个任务
166 for task_id in tqdm.tqdm(range(num_tasks_in_suite)):
167     # 获取当前任务
168     task = task_suite.get_task(task_id)
169     # 获取任务的初始状态 (仿真环境复位用)
170     initial_states = task_suite.get_task_init_states(task_id)
171     # 初始化LIBERO环境 + 获取任务自然语言描述
172     env, task_description = get_libero_env(task, cfg.model_family, re
solution=256)
173
174     # 当前任务的统计变量
175     task_episodes, task_successes = 0, 0
176
177     # 对当前任务执行多次尝试 (默认50次)
178     for episode_idx in tqdm.tqdm(range(cfg.num_trials_per_task)):
179         print(f"\n任务: {task_description}")
180         log_file.write(f"\n任务: {task_description}\n")
181
182         # 复位仿真环境
183         env.reset()
184         # 设置环境初始状态
185         obs = env.set_init_state(initial_states[episode_idx])
186
187         # 初始化步数、图像缓存列表
188         t = 0
189         replay_images = []
190
191         # ===== 根据任务集设置最大执行步数 =====
192         =====
193         if cfg.task_suite_name == "libero_spatial":
194             max_steps = 220
195         elif cfg.task_suite_name == "libero_object":
196             max_steps = 280
197         elif cfg.task_suite_name == "libero_goal":
198             max_steps = 300
199         elif cfg.task_suite_name == "libero_10":
200             max_steps = 520
201         elif cfg.task_suite_name == "libero_90":
202             max_steps = 400
203
204         print(f"开始第 {task_episodes+1} 次尝试...")
205         log_file.write(f"开始第 {task_episodes+1} 次尝试...\n")
206
207         # ===== 单轮任务执行循环 =====
208         while t < max_steps + cfg.num_steps_wait:

```

```

208         try:
209             # 前N步：执行空动作，让仿真物体稳定下落
210             if t < cfg.num_steps_wait:
211                 obs, reward, done, info = env.step(get_libero_dum
my_action(cfg.model_family))
212                 t += 1
213                 continue
214
215             # 1. 获取预处理后的仿真图像
216             img = get_libero_image(obs, resize_size)
217             # 缓存图像，用于生成演示视频
218             replay_images.append(img)
219
220             # 2. 构造模型输入：图像 + 机器人状态
221             observation = {
222                 "full_image": img,
223                 "state": np.concatenate(
224                     (obs["robot0_eef_pos"], quat2axisangle(obs["r
obot0_eef_quat"]), obs["robot0_gripper_qpos"])
225                     ),
226             }
227
228             # 3. OpenVLA模型推理：根据图像+语言指令输出机器人动作
229             action = get_action(
230                 cfg,
231                 model,
232                 observation,
233                 task_description,
234                 processor=processor,
235             )
236
237             # 4. gripper (夹爪) 动作归一化：[0,1] → [-1,+1] (符合环境
要求)
238             action = normalize_gripper_action(action, binarize=True)
239
240             # 5. OpenVLA专用：反转夹爪动作符号 (对齐数据集格式)
241             if cfg.model_family == "openvla":
242                 action = invert_gripper_action(action)
243
244             # 6. 执行动作：把模型输出的动作发送到仿真环境
245             obs, reward, done, info = env.step(action.tolist())
246
247             # 如果任务完成 (成功)，计数并退出当前循环
248             if done:
249                 task_successes += 1
250                 total_successes += 1
251                 break

```

```

252         t += 1
253
254     # 捕获异常并记录日志
255     except Exception as e:
256         print(f"捕获异常: {e}")
257         log_file.write(f"捕获异常: {e}\n")
258         break
259
260     # 更新计数
261     task_episodes += 1
262     total_episodes += 1
263
264     # ===== 保存本轮尝试的视频 =====
265     save_rollout_video(
266         replay_images, total_episodes, success=done, task_descrip
267         tion=task_description, log_file=log_file
268     )
269
270     # ===== 打印并记录当前结果 =====
271     success_rate = total_successes / total_episodes * 100
272     print(f"成功: {done}")
273     print(f"已完成尝试次数: {total_episodes}")
274     print(f"成功次数: {total_successes} ({success_rate:.1f}%)")
275     log_file.write(f"成功: {done}\n")
276     log_file.write(f"已完成尝试次数: {total_episodes}\n")
277     log_file.write(f"成功次数: {total_successes} ({success_rate:.1
278     f}%)\\n")
279     log_file.flush() # 立即写入文件
280
281     # ===== 当前任务完成, 输出最终成功率 =====
282     task_success_rate = float(task_successes) / float(task_episodes)
283     total_success_rate = float(total_successes) / float(total_episode
284     s)
285     print(f"当前任务成功率: {task_success_rate}")
286     print(f"总体成功率: {total_success_rate}")
287     log_file.write(f"当前任务成功率: {task_success_rate}\\n")
288     log_file.write(f"总体成功率: {total_success_rate}\\n")
289     log_file.flush()
290
291     # 上传当前任务结果到Wandb
292     if cfg.use_wandb:
293         wandb.log(
294             {
295                 f"success_rate/{task_description}": task_success_rate
296                 ,
297                 f"num_episodes/{task_description}": task_episodes,
298             }

```

```

295         )
296
297     # ===== 全部任务评估完成 =====
298     log_file.close() # 关闭本地日志
299
300     # 上传最终总指标到Wandb
301     if cfg.use_wandb:
302         wandb.log(
303             {
304                 "success_rate/total": float(total_successes) / float(total_episodes),
305                 "num_episodes/total": total_episodes,
306             }
307         )
308         wandb.save(local_log_filepath) # 保存日志文件到云端
309
310
311     # ===== 程序入口 =====
312     if __name__ == "__main__":
313         eval_libero()

```

(3)核心代码执行流程（必看）

我用最清晰的逻辑帮你梳理代码运行的完整步骤：

1. 配置解析

- 读取命令行参数 → 生成 `GenerateConfig` 配置对象
- 校验关键参数（模型路径、量化、中心裁剪）

2. 初始化

- 固定随机种子 → 保证复现性
- 加载OpenVLA模型 + 处理器
- 创建本地日志文件 + 初始化Wandb

3. 加载LIBERO环境

- 加载指定任务集（spatial/object/goal/10/90）
- 获取所有任务、初始状态、最大执行步数

4. 批量评估（核心循环）

- 1 遍历每个任务 → 执行50次尝试 → 每次尝试：
- 2 1. 复位环境 → 等待物体稳定
- 3 2. 获取相机图像 → 预处理
- 4 3. OpenVLA模型推理 → 输出机器人动作
- 5 4. 动作格式转换（夹爪归一化+反转）
- 6 5. 执行动作 → 检查是否完成任务
- 7 6. 保存视频、记录日志、统计成功率

5. 结果输出

- 本地保存日志+视频
- Wandbox上传成功率曲线
- 打印最终评估报告

(4)关键技术细节（OpenVLA专家解读）

1. center_crop=True 强制要求

- 模型用随机裁剪增强训练时，评估必须中心裁剪，否则精度大幅下降

2. 动作归一化/反转

- OpenVLA数据集格式与LIBERO环境不一致，必须做 `normalize_gripper_action` + `invert_gripper_action`

3. num_steps_wait

- 仿真环境中物体会下落，前10步空动作保证环境稳定

4. unnorm_key

- OpenVLA模型必须加载对应数据集的统计信息，否则动作尺度错误

(5)总结

1. 这是OpenVLA在LIBERO上的标准评估脚本，可直接运行
2. 代码结构 = 配置 → 初始化 → 环境加载 → 批量评估 → 结果保存
3. 核心是模型推理 → 动作处理 → 仿真执行 → 成功率统计

- 4. 注释已覆盖每一行代码的作用、参数含义、技术细节、异常处理
- 5. 运行时只需修改： `pretrained_checkpoint` 、 `task_suite_name` 、 `wandb` 信息

五、复现过程中一些补充知识的学习

1. 什么是octo模型？

Octo 是当前机器人领域基于大规模多机数据训练的开源通用策略模型，核心定位是做机器人的“通用控制大脑”，主打跨机器人、跨任务、跨传感器的强泛化与低成本微调。

一、Octo 核心定位与背景

- 全称：Octo: An Open-Source Generalist Robot Policy（开源通用机器人策略）
- 训练基础：基于 Open X-Embodiment 数据集，用 80 万条真实机器人操作轨迹 预训练，覆盖 25+ 数据集、多种机械臂（UR5、Franka、WidowX 等）与任务（抓取、开关、擦拭、倒液体等）。
- 核心目标：打破“一机器人一策略、一任务一模型”的传统，实现一次预训练、多机复用、少量数据快速适配新场景。
- 模型版本：提供 Octo-Small（27M）、Octo-Base（93M），兼顾速度与能力。

二、核心技术架构（Transformer + 扩散策略 + 模块化设计）

Octo 是 Transformer 架构 + 扩散策略（Diffusion Policy）的结合体，最大创新是输入输出解耦、模块化、可插拔。

1. 输入层：多模态统一 Token 化

- 任务输入：自然语言指令（T5 编码）、目标图像（CNN 编码）。
- 观测输入：多相机 RGB（腕部/全局）、本体 proprio 信息、历史观测序列。
- 统一处理：所有输入转为 Token，按时序/模态拼接，支持任意增减传感器/相机。

2. 主干：因果 Transformer + Readout Token（核心创新）

- 采用因果自注意力：观测 Token 只能关注当前及更早时间步，保证时序合理性。
- Readout Token：将“输入编码”与“动作生成”解耦，动作头只从 Readout Token 提取信息，换机

器人/换动作空间无需重训主干，仅需微调轻量 Adapter。

3. 输出层：扩散策略生成平滑动作序列

- 用 扩散模型（Diffusion） 输出连续、平滑的动作序列（而非单步动作），天然解决 OpenVLA 等单步预测的抖动、轨迹不平滑问题。
- 输出可适配：末端位姿（x/y/z/rx/ry/rz）、夹爪开合、关节角度等多种动作空间。

三、Octo 三大核心优势（区别于 OpenVLA/RT-1-X）

1. 极致灵活性：跨机器人/传感器/动作空间

- 支持任意相机组合（1/2/3 个相机，腕部/全局）、任意关节数（6/7 轴）、任意动作定义。
- 换机器人时：仅需加轻量 Adapter + 少量数据微调（约 100 条轨迹，普通 GPU 几小时），无需重训整个模型。

2. 动作质量：扩散策略天然平滑、低抖动

- 输出完整短轨迹（如 5—10 步连续动作），而非单步指令，轨迹平滑、执行稳定，大幅减少抖动。
- 对比：OpenVLA 多输出单步，需额外后处理/闭环；Octo 内置扩散，开箱即用更稳。

3. 开源与易用：完整生态、开箱即用

- 完全开源：模型权重、训练代码、推理脚本、数据 pipeline 全部开放。
- 开箱即用：支持语言/目标图像双指令，零样本控制预训练见过的机器人；微调后适配新机型。

四、Octo 与 OpenVLA/RT-1-X 关键对比

维度	Octo	OpenVLA	RT-1-X
架构	Transformer + 扩散策略	Transformer + 单步回归	Transformer + 行为克隆
动作输出	平滑轨迹序列（扩散生成）	单步位姿/关节	单步离散/连续动作
灵活性	极高：可插拔传感器/动作空间	中：固定输入输出	低：绑定训练时配置

微调成本	极低：100 条数据+几小时	中：需较多数据	高：重训成本大
抖动控制	内置扩散，天然平滑	需额外后处理/闭环	需工程优化
开源性	完全开源	部分开源	闭源/受限

五、落地价值（对你的机械臂抓取场景）

- 1. **动作更稳**：扩散生成轨迹，**无需额外扩散模型做后处理**，直接输出平滑指令，解决抖动。
- 2. **快速适配**：若换机械臂/相机，**少量数据微调即可**，不用从零训练。
- 3. **多任务兼容**：一套模型支持抓取、放置、开关等，支持语言指令（“抓取红色杯子”）与目标图像双模式。
- 4. **坐标系友好**：预训练数据多标注**机械臂基坐标系**位姿，输出可直接下发，减少手眼标定依赖。

六、总结

Octo 是大规模多机数据训练 + Transformer + 扩散策略 + 模块化设计的集大成者，是当前通用机器人策略的标杆之一。它解决了传统模型“绑定配置、难迁移、动作抖”的痛点，非常适合需要**跨平台、多任务、稳定抓取**的机械臂项目。

需要我给你整理一份 Octo 快速部署与推理的最简代码示例（含加载预训练模型、输入语言指令、输出平滑动作序列）吗？

2.什么是FSDP?

一、FSDP 核心定义（大白话版）

FSDP（Fully Sharded Data Parallel，完全分片数据并行）是 **PyTorch 官方推出的大模型训练加速/显存优化技术**，核心作用是：

把超大模型（比如 Octo/OpenVLA 这类亿级/十亿级参数的 VLA 模型）的**参数、梯度、优化器状态** 拆成小块，分散到多块 GPU 上，让单卡显存装不下的大模型，能在多卡集群里高效训练。

简单说：**单卡装不下一个大模型 → FSDP 把模型“拆碎”分给多卡 → 多卡一起训练 → 训练完再拼回去。**

二、为什么需要 FSDP?（结合你的场景）

你关注的 Octo/OpenVLA 这类 VLA 模型，参数动辄几千万到几亿：

- 单张消费级 GPU（比如 24G 显存）根本装不下完整模型；
 - 就算装下，训练时还要存梯度、优化器状态，显存直接爆掉；
- FSDP 就是为解决“大模型训练显存不够、多卡协同低效”而生的。

三、FSDP 核心原理（极简拆解）

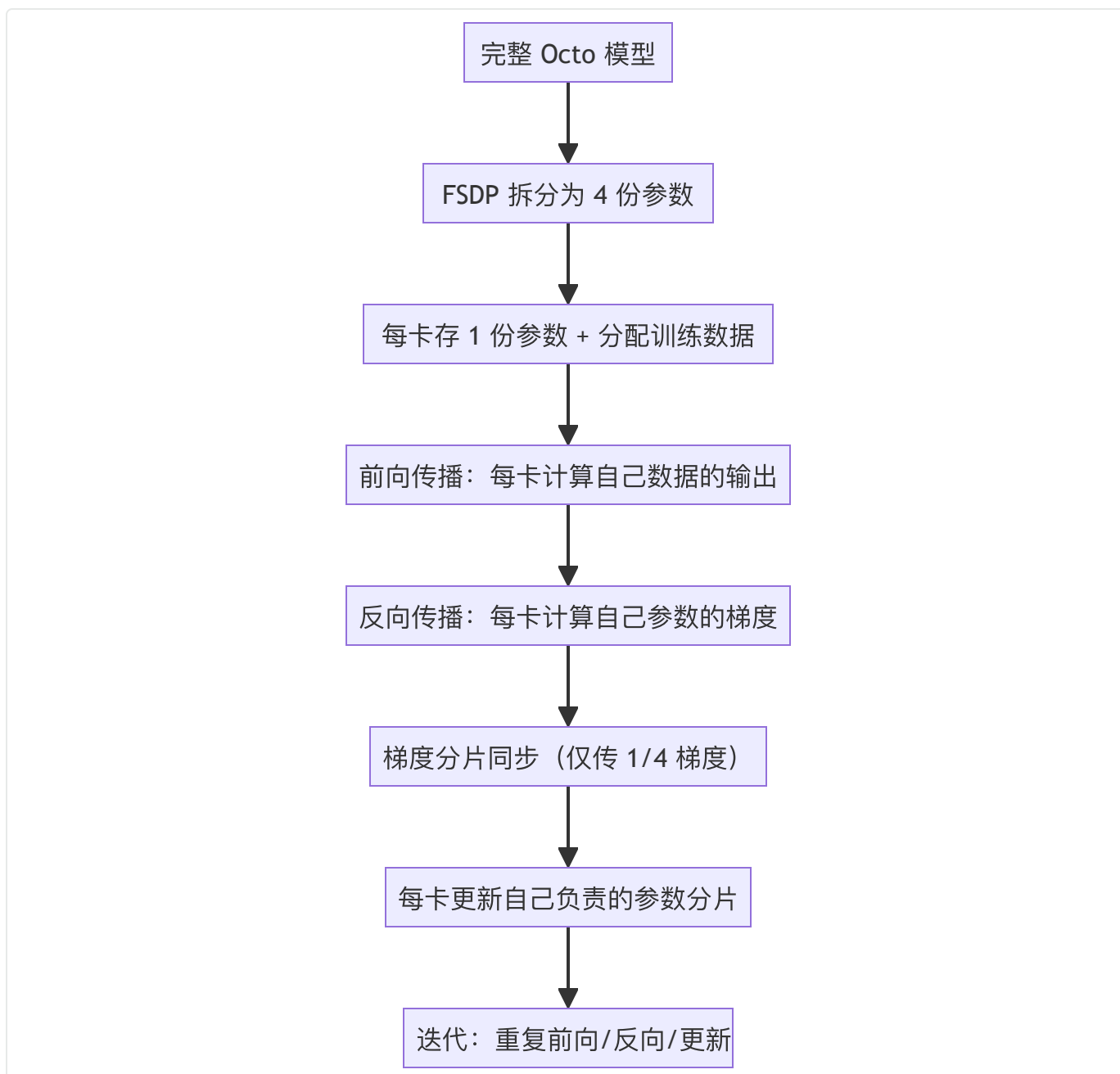
1. 核心动作：“分片”+“按需通信”

- **模型参数分片**：把模型的每一层参数拆成 N 份（N=GPU 数量），每块 GPU 只存 1/N 的参数；
比如 4 卡训练 Octo-Base（93M 参数），每卡只存 ~23M 参数，显存占用直接降 4 倍。
- **梯度分片**：反向传播时，每卡只计算自己负责参数的梯度，再通过通信同步给其他卡；
- **优化器状态分片**：Adam 这类优化器的动量、方差等状态，也跟着参数分片存储，进一步省显存。

2. 关键区别：对比传统 Data Parallel（DP）

维度	传统 DP（数据并行）	FSDP（完全分片数据并行）
单卡存储内容	完整模型参数 + 部分数据的梯度	1/N 模型参数 + 对应梯度 + 对应优化器状态
显存占用	高（单卡存完整模型）	低（仅存 1/N）
支持模型规模	小（单卡能装下的模型）	大（单卡装不下的超大模型）
多卡通信开销	高（梯度全量同步）	低（仅同步分片梯度）

3. 训练流程（以 4 卡训练 Octo 为例）



四、FSDP 在 Octo/OpenVLA 中的应用（对你的价值）

1. Octo/OpenVLA 训练必用 FSDP 的原因

- Octo-Base (93M 参数) / OpenVLA (千万级参数) :
若用 4 卡训练，FSDP 可让单卡显存占用从“爆显存”降到 ~8G（24G 消费级 GPU 就能跑）；
- 若微调更大的 VLA 模型（比如 7B 级），FSDP 是唯一能让多卡协同训练的核心技术。

2. 对你的落地价值

- **低成本微调**：不用买超高端的 A100（80G），用 4 张 3090（24G）就能微调 Octo/OpenVLA；
- **训练速度快**：分片通信开销低，多卡训练效率比传统 DP 高 30%+；
- **官方支持好**：PyTorch 原生集成，Octo/OpenVLA 的开源代码里直接内置 FSDP 配置，开箱即用。

五、FSDP 关键配置（极简实操）

以 Octo 训练代码为例，FSDP 核心配置就几行：

```
1  import torch
2  from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
3
4  # 1. 初始化分布式环境（多卡通信）
5  torch.distributed.init_process_group(backend="nccl")
6
7  # 2. 加载 Octo 模型（未分片）
8  model = OctoModel.from_pretrained("octo-base")
9
10 # 3. 用 FSDP 包装模型（核心！）
11 model = FSDP(
12     model,
13     sharding_strategy=torch.distributed.fsdp.ShardingStrategy.FULL_SHARD,
14     # 完全分片
15     cpu_offload=False, # 消费级GPU建议关闭，专业卡可开
16     auto_wrap_policy=lambda m: isinstance(m, torch.nn.TransformerLayer)
17     # 按 Transformer 层分片
18 )
19
20 # 4. 正常训练即可，FSDP 自动处理多卡协同
21 for data in dataloader:
22     loss = model(data)
23     loss.backward()
24     optimizer.step()
```

六、核心总结

1. **FSDP 本质**：把大模型拆成小块分给多卡，降低单卡显存占用，实现超大模型的多卡高效训练；
2. **核心优势**：显存占用低、通信开销小、支持超大规模模型；
3. **对你的价值**：用消费级 GPU 就能微调 Octo/OpenVLA，不用依赖高端算力，大幅降低训练成本。

如果需要，我可以给你整理 **Octo 基于 FSDP 的多卡微调完整代码模板**（含环境配置、显存优化、训练脚本），直接就能跑。

3.什么是ManiSkill2：通用机器人操作的大规模仿真基准

ManiSkill2 和 LIBERO 都是机器人操作/具身智能领域的开源仿真基准平台，核心用途是：在虚拟环境中大规模训练、测试、评估机器人抓取/操作策略，加速算法从仿真到真机（Sim2Real）的落地。下面分别详细介绍，并做对比。

1. 定位与核心目标

ManiSkill2 是面向**通用机器人操作技能（Generalizable Manipulation）**的统一仿真基准，由 UC San Diego、斯坦福、伯克利、上海AI实验室等联合开发，基于 SAPIEN 物理引擎。

一句话定位：做“机器人操作领域的 ImageNet”，提供海量、多样、可泛化的仿真任务与数据，用于训练/评估 Octo、Diffusion Policy、OpenVLA 等通用策略模型。

2. 核心能力与用途

- **任务覆盖极广：**20+ 任务系列、2000+ 物体、400 万+ 演示帧，覆盖：
 - 基础操作：PickCube、PickSingleYCB、PegInsertion、PlugCharger、TurnFaucet 等
 - 复杂操作：装配、倒水、悬挂、挖掘、写字、避障、双臂协作、软体操作
 - 机器人类型：单臂/双臂、固定/移动基座、夹爪/灵巧手
- **支持全流程算法研发：**
 - 强化学习（RL）、模仿学习（IL/BC）、行为克隆、Diffusion Policy 等
 - 感知（RGBD/点云）+ 规划 + 控制的端到端训练
 - 泛化性测试：物体级、场景级、任务级泛化能力评估
- **高效仿真：**单 GPU + 多进程可达到 **2000+ FPS**，支持大规模数据生成与快速迭代
- **社区与竞赛：**CVPR 2023 举办 ManiSkill2 挑战赛，是当前机器人操作领域最主流的基准之一

3. 典型应用场景

- 训练 Octo/OpenVLA/Diffusion Policy 等通用抓取策略

- 验证 Sim2Real 迁移能力（仿真训练 → 真机部署）
 - 对比不同算法在复杂操作任务上的泛化性与成功率
 - 生成大规模演示数据用于模仿学习
-

4. 什么是LIBERO：面向终身/多任务学习的机器人操作基准

1. 定位与核心目标

LIBERO 是专为机器人终身学习（Lifelong Learning）、多任务知识迁移、持续学习设计的仿真基准，由 MIT、斯坦福等团队开发，基于 PyBullet 物理引擎。

一句话定位：聚焦“机器人如何在不断学习新任务时，不遗忘旧知识、有效迁移已有技能”，解决多任务/终身学习的评估难题。

2. 核心能力与用途

- 四大任务套件（按泛化维度划分）：
 - LIBERO-Spatial：空间位置/姿态变化（同一物体不同摆放）
 - LIBERO-Object：物体类别/外观变化（不同物体、不同纹理）
 - LIBERO-Goal：目标语义变化（同一任务不同指令，如“把红块放蓝块上”/“把蓝块放红块上”）
 - LIBERO-100：100 个多样化任务的综合套件，覆盖抓取、放置、堆叠、开门等
- 聚焦知识迁移与持续学习：
 - 评估模型在连续学习新任务时的“灾难性遗忘”
 - 测试跨任务知识复用能力（学会抓杯子 → 快速学会抓碗）
 - 支持少样本/零样本泛化到新任务/新物体
- 轻量化、易扩展：
 - 基于 PyBullet，部署简单、计算开销低
 - 提供统一 API，方便接入各类策略模型（BC、RL、Diffusion 等）

3. 典型应用场景

- 研究多任务/终身学习算法（如持续学习、元学习、知识蒸馏）
- 评估 Octo/OpenVLA 等模型在“不断学习新任务”时的稳定性与迁移效率

- 测试策略在语义/空间/物体三个维度的泛化边界
- 构建可持续进化的机器人操作系统

5.ManiSkill2 vs LIBERO：核心区别对比

维度	ManiSkill2	LIBERO
核心定位	通用操作技能、大规模泛化、Sim2Real	多任务/终身学习、知识迁移、持续学习
物理引擎	SAPIEN（高精度、动态仿真强）	PyBullet（轻量化、易部署）
任务规模	20+ 系列、2000+ 物体、400 万+ 帧	4 大套件、100+ 任务、聚焦泛化维度
核心评估点	操作成功率、物体/场景泛化、Sim2Real 迁移	知识迁移效率、持续学习、灾难性遗忘
典型模型	Octo、Diffusion Policy、OpenVLA、ACT	终身学习算法、元学习、多任务策略
社区热度	极高（CVPR 竞赛、工业界广泛使用）	高（学术圈多任务/终身学习主流）
优势	任务丰富、仿真高效、Sim2Real 友好	聚焦知识迁移、轻量化、易扩展

1. ManiSkill2 更适合：

- 训练/测试通用抓取/操作策略（如 Octo、Diffusion Policy）
- 做大规模数据生成与Sim2Real 迁移验证
- 对比不同模型在复杂操作任务上的泛化能力

2. LIBERO 更适合：

- 研究多任务/终身学习，让机器人“学会一个、举一反三”
- 测试模型在不断学习新任务时的稳定性与遗忘程度
- 聚焦知识迁移与少样本泛化

6.什么是adapter适配器?

```
# -----  
# 4) 可选: 加载 LoRA 权重  
# -----  
# 如果你有 lora_checkpoint (比如自己微调的 adapter), 可以在这里叠加到 base 模型上  
if config.lora_checkpoint:  
    model = PeftModel.from_pretrained(model, config.lora_checkpoint)  
    print(f"Loaded lora weights from {config.lora_checkpoint}")
```

大模型中 Adapter 的核心含义

在大模型 (LLM/VLM) 领域, **Adapter (适配器)** 是一种「轻量化微调/适配」技术, 核心思想是: **不修改大模型的主干参数 (冻结)**, 仅在模型中插入少量可训练的小型网络模块 (**Adapter层**), 通过训练这些小模块让大模型适配特定任务 (如翻译、问答、视觉-语言对齐), 以此大幅降低显存/计算开销, 同时避免大模型过拟合。

可以用一个通俗的比喻理解:

- 大模型本身是一台「通用的超级计算机」, 能处理各种基础任务, 但对「特定场景 (如工业数据分析)」不擅长;
- Adapter 就是给这台超级计算机加装的「专用插件/扩展卡」, 不用改造计算机本身, 只需调试插件, 就能让它高效处理特定任务。

Adapter 的核心设计与作用

1. 为什么需要 Adapter?

大模型 (如7B/13B/70B参数) 直接全量微调的痛点:

- 显存开销大: 7B模型全量微调 (bf16) 需至少30GB显存, 普通显卡无法支撑;
- 计算成本高: 全量参数更新, 训练时间/电费成本极高;
- 易遗忘原有知识 (灾难性遗忘): 微调后模型可能丢失通用能力;
- 多任务适配难: 每个任务都要保存一份完整模型权重, 存储成本高。

Adapter 完美解决这些问题:

- 可训练参数仅占原模型的 0.1%~5% (比如7B模型的Adapter仅几十万到几百万参数);

- 显存占用降低50%以上，消费级显卡（如4090）也能微调；
- 冻结主干模型，保留通用能力，仅适配特定任务；
- 多任务可通过「切换Adapter权重」实现，只需保存多个小Adapter文件（几MB~几十MB），而非完整模型。

2. Adapter 的典型结构（以LLM为例）

Adapter 通常插入在 Transformer 层的「注意力层」和「前馈层」之间，核心结构是「降维-非线性变换-升维」（也叫 bottleneck 结构），示例如下：

```

1  # 简化的 Adapter 结构伪代码
2  class Adapter(nn.Module):
3      def __init__(self, hidden_dim, adapter_dim=64):
4          super().__init__()
5          # 降维: 把模型高维特征（如4096维）压缩到低维（如64维）
6          self.down_proj = nn.Linear(hidden_dim, adapter_dim)
7          # 非线性激活
8          self.act = nn.ReLU()
9          # 升维: 还原回原维度，与主干特征相加
10         self.up_proj = nn.Linear(adapter_dim, hidden_dim)
11         # 初始化: 保证初始时Adapter输出接近0，不影响主干模型
12         nn.init.zeros_(self.up_proj.weight)
13         nn.init.zeros_(self.up_proj.bias)
14
15     def forward(self, x):
16         # x: 主干模型的特征 ([batch, seq_len, hidden_dim])
17         adapter_out = self.up_proj(self.act(self.down_proj(x)))
18         # 残差连接: Adapter输出叠加到原特征上
19         return x + adapter_out

```

在 Transformer 层中的插入逻辑：

```

1  # Transformer层 + Adapter 简化版
2  class TransformerLayerWithAdapter(nn.Module):
3      def __init__(self, hidden_dim):
4          super().__init__()
5          self.attention = AttentionLayer(hidden_dim) # 主干注意力层 (冻结)
6          self.ffn = FeedForwardLayer(hidden_dim)    # 主干前馈层 (冻结)
7          self.adapter = Adapter(hidden_dim)         # Adapter层 (可训练)
8
9      def forward(self, x):
10         x = self.attention(x) + x # 主干注意力计算 (冻结)
11         x = self.adapter(x)      # 插入Adapter计算 (训练)
12         x = self.ffn(x) + x      # 主干前馈计算 (冻结)
13         return x

```

3. 常见的 Adapter 变体

不同场景有不同的 Adapter 设计，核心思路一致，仅结构细节不同：

- **LoRA (Low-Rank Adaptation)**：最流行的变体，本质是对注意力层的权重矩阵做低秩分解，仅训练分解后的小矩阵（可看作特殊的Adapter）；
- **IA3**：对特征做逐元素缩放，参数更少；
- **Parallel Adapter**：与主干层并行计算，而非串行插入；
- **Visual Adapter**：在VLM（视觉-语言模型）中，用于适配不同视觉特征（如CLIP/ViT输出）到语言模型的维度。

4. Adapter 的使用流程（新手友好）

以 Hugging Face Transformers 为例，使用 Adapter 的核心步骤：

```

1  from peft import get_peft_model, LoraConfig # PEFT库: 专门实现Adapter/LoRA
2
3  # 1. 加载预训练大模型 (冻结所有主干参数)
4  model = AutoModelForCausalLM.from_pretrained("llama-7b", torch_dtype=torch
        .bfloat16)
5  for param in model.parameters():
6      param.requires_grad = False # 冻结主干
7
8  # 2. 配置Adapter (以LoRA为例)
9  peft_config = LoraConfig(
10     r=8, # 低秩维度 (越小参数越少)
11     lora_alpha=32,
12     target_modules=["q_proj", "v_proj"], # 仅对注意力层的q/v矩阵加Adapter
13     lora_dropout=0.05,
14     bias="none",
15     task_type="CAUSAL_LM"
16 )
17
18 # 3. 给模型添加Adapter层
19 model = get_peft_model(model, peft_config)
20 # 查看可训练参数: 仅占原模型的0.1%左右
21 model.print_trainable_parameters() # 输出: trainable params: 1,234,567 ||
    all params: 7,000,000,000 || trainable%: 0.0176
22
23 # 4. 训练模型: 仅更新Adapter参数
24 trainer = Trainer(model=model, ...)
25 trainer.train()
26
27 # 5. 保存/加载Adapter权重 (仅几MB)
28 model.save_pretrained("llama-7b-adapter-qa")
29 # 加载: 先加载原模型, 再加载Adapter
30 model = AutoModelForCausalLM.from_pretrained("llama-7b")
31 model = PeftModel.from_pretrained(model, "llama-7b-adapter-qa")

```

总结

1. Adapter 是大模型的轻量化适配技术: 冻结主干参数, 仅训练插入的小型模块, 大幅降低微调成本;
2. 核心优势: 显存/计算开销低、避免灾难性遗忘、多任务适配灵活 (仅切换Adapter权重);
3. 常见变体: LoRA (最流行)、IA3等, 均基于「少量可训练参数适配特定任务」的核心思想;
4. 适用场景: 大模型的下游任务微调 (如问答、翻译、VLM视觉对齐)、低资源设备的大模型适配。

7.什么是wandb?

(1) 参考学习博客

- <https://zhuanlan.zhihu.com/p/1996527961839526185>
- <https://github.com/datawhalechina/diy-llm/blob/main/docs/chapter1/wandb%E4%BD%BF%E7%94%A8%E4%BB%8B%E7%BB%8D.md>

六、base模型直接跑libero的结果

- 虽然能动，但是成功率为0基本上都是在瞎动

七、openvla的代码结构和其推理测试脚本

1.模型代码结构

代码目录树

Tree

openvla/

- prismatic/
 - conf/
 - models/
 - load.py
 - materialize.py
 - registry.py
 - backbones/
 - vision/
 - base_vision.py
 - llm/
 - base_llm.py
 - prompting/
 - base_prompter.py
 - vlas/
 - openvla.py
 - vlms/
 - base_vlm.py
 - prismatic.py
 - overwatch/
 - overwatch.py
 - vla/
 - action_tokenizer.py
 - vla-scripts/
 - deploy.py
 - finetune.py
 - train.py

核心工具

配置管理

模型架构

模型加载工具

模型实例化工具

模型注册表

主干网络

视觉主干网络

VisionBackbone

语言模型主干网络

LLMBackbone

提示构建器

提示构建器基类

OpenVLA实现

抽象VLM基类 (VLM)

PrismaticVLM实现

日志系统实现

ActionTokenizer

训练、微调 and 部署VLAs的核心脚本

REST API部署

LoRA微调

完整训练

类继承关系图

VLM (抽象基类)

- PrismaticVLM (继承自VLM)
 - 组成部件:
 - VisionBackbone (抽象基类)
 - CLIPViT / DINOViT / SigLIPViT 等 (VisionBackbone)
 - LLMBackbone (抽象基类)
 - Llama2Backbone 等 (继承自LLMBackbone)
 - 包含: Llama2ChatPrompter
 - Projector Layers
 - Linear Projector
 - MLP Projector
 - Fused MLP Projector
 - OpenVLA (继承自PrismaticVLM)
 - 包含: ActionTokenizer (动作预测)

VLM作为抽象基类，PrismaticVLM是具体实现。
OpenVLA进一步扩展，支持动作预测。

55

2.推理的测试脚本（图片+指令——>7D的动作）

模型推理代码

inference

```
via_inference.py x image.png A
via_inference.py > ...
1 from transformers import AutoModelForVision2Seq, AutoProcessor
2 from PIL import Image
3
4 import torch
5
6 # Load Processor & VLA
7 model_path = './model'
8
9 processor = AutoProcessor.from_pretrained(model_path, trust_remote_code=True)
10 vla = AutoModelForVision2Seq.from_pretrained(
11     model_path,
12     # attn_implementation="flash_attention_2",
13     torch_dtype=torch.bfloat16,
14     low_cpu_mem_usage=True,
15     trust_remote_code=True
16 ).to("cuda:0")
17
18 image = Image.open("./image.png")
19
20 instruction = "pick up the green grape"
21 prompt = f"In: What action should the robot take to {instruction}? \nOut:"
22
23 # Predict Action (7-DoF; un-normalize for BridgeData V2)
24 inputs = processor(prompt, image).to("cuda:0", dtype=torch.bfloat16)
25 action = vla.predict_action(**inputs, unnorm_key="bridge_orig", do_sample=False)
26
27 print("Predicted Action:", action)
```

1. 加载预训练模型 (VLA)

2. 输入：图像 + 文本指令 ("捡起青葡萄")

3. 模型预测 → 输出7维动作向量



6D 笛卡尔末端执行器运动（对应于相对位姿的变化）+ 1D 控制夹持器开合

八、openvla的推理和执行频率篇匹配

OpenVLA 默认是推理一次、执行一次的单步闭环模式，但它也支持动作分块（Action Chunking）等优化来提升效率；其推理速度足以满足常规机器人实时控制，不会太慢。

一、默认执行模式：单步闭环（推理→执行→再推理）

- 核心流程：

- 机器人获取当前图像 + 语言指令
- OpenVLA 推理输出一组7D动作（3位置+4姿态/四元数）

- c. 底层控制器执行该动作（通常对应约 0.1—0.2秒 的运动）
- d. 采集新图像，回到步骤2，循环直到任务完成
- 为什么这么做：
 - 单步预测误差小，可通过实时观测修正，容错性强
 - 动作平滑、响应快，适合大多数抓取、摆放、装配等操作任务

二、速度与延迟：满足实时控制，不慢

- 标准 OpenVLA（7B，无加速）：
 - RTX 4090：6 Hz (≈ 167 ms/步)
 - Jetson Orin：10 Hz (≈ 100 ms/步)
 - 机器人实时控制的黄金阈值 ≥ 5 Hz，OpenVLA 达标
- OpenVLA-OFT（优化版，动作分块+并行解码）：
 - A100：77.9 Hz (≈ 12.8 ms/步)
 - 双臂机器人 ALOHA：稳定 25 Hz 工业级控制
- 结论：
 - 常规任务：6—10 Hz 完全够用，不会卡顿
 - 高动态任务（如快速避障、接球）：可用 OFT/动作分块 提升到 25—70+ Hz

三、如何解决“单步慢”的问题（优化方案）

1. 动作分块（Action Chunking）
 - 一次推理输出未来N步动作序列（如8步/16步）
 - 执行时按序列连续运动，减少推理次数、提升平滑度
2. 并行解码（Parallel Decoding）
 - 放弃自回归，一次前向传播生成整段动作块
 - 吞吐量提升 26—43倍，从 1.8 Hz $\rightarrow$ 77.9 Hz
3. 量化与硬件优化
 - INT8/INT4 量化、bfloat16 精度，显存减半、速度提升
 - 适配 Jetson Orin、Thor 等端侧机器人平台

四、一句话总结

- 默认：单步闭环（推理→执行），6—10 Hz，满足实时
- 优化：动作分块+并行解码，可达 25—70+ Hz，适配高动态任务

需要我帮你整理一份 OpenVLA 不同硬件/优化方案的推理帧率与延迟对比表，方便你选型吗？

九、运行官方测评程序的终端命令

```
Python |  
1 python experiments/robot/libero/run_libero_eval.py  
2 --model_family openvla  
3 --pretrained_checkpoint /root/data/openvla_work/runs/openvla-7b+libero_spatial_no_noops+b8+lr-0.0005+lora-r8+dropout-0.05--image_aug--4000_chkpt  
4 --task_suite_name libero_spatial  
5 --center_crop=true  
6 --num_trials_per_task 20  
7 --run_id_note "base-model-finetuned-4000"  
8 --use_wandb=true  
9 --seed 7
```